Laboratory Record

CSA4029 Reinforcement and Representation Learning

B21+E14+E22, BL2025260500511,

Winter Semester:2025-2026

Name of the student: Aditi Gupta

Register Number: 23BAI10968

Exp No: 3	Atari (DQN)
31.03.26	

Aim: A Deep Q-Network (DQN) for Atari games (like Breakout or Pong) is a significant step up from basic Q-Learning. Because the "state" is a raw image (pixels), we swap the Q-Table for a Convolutional Neural Network (CNN).

Procedure:

Step 1: Preprocessing the "Vision"

Atari frames are colorful and large (210x160 pixels). To make the math efficient, we transform the raw data:

- Grayscale & Resize: We convert frames to black and white and shrink them to 84x84 pixels.
- Frame Stacking: A single image doesn't show motion (you can't tell if a ball is moving left or right). We stack the last 4 frames together as a single input so the network can "see" velocity and direction.

Step 2: The Convolutional "Feature Finder"

The agent doesn't receive coordinates for the ball; it receives a grid of numbers representing brightness.

- The Convolutional Layers act like filters. They scan the image to detect edges, then shapes, then objects (like the paddle or the ball).
- The final Fully Connected Layers take these detected objects and decide which action (Up, Down, Fire) will lead to the highest score.

Step 3: Experience Replay (The "Memory Bank")

If the agent learned only from its most recent move, it would forget old lessons and get "stuck" in a loop of similar data.

- We store thousands of past experiences—(state, action, reward, next_state)—in a Replay Buffer.
- During training, we pull a random batch of 32 or 64 experiences from this bank.
- Why? This breaks the correlation between consecutive frames and ensures the agent learns from a diverse set of past successes and failures.

Step 4: The Double-Network Strategy

DQN uses two separate neural networks to solve the "moving target" problem:

1. Policy Network: This is the "active" brain. It is updated every single step to get better at predicting rewards.
2. Target Network: This is a "snapshot" or a copy of the Policy Network. It stays frozen for, say, 1,000 steps.

- The Logic: We use the *Target Network* to calculate the "ideal" reward for the next state. Because it's frozen, the goalpost stays still long enough for the Policy Network to actually learn how to kick the ball toward it.

Step 5: Calculating the Loss (Backpropagation)

The agent calculates the difference (Loss) between:

- Prediction: What the Policy Network guessed the reward would be.
- Target: The actual reward received + the discounted future reward (calculated by the Target Network).

Using an optimizer like Adam, the agent adjusts the weights of the neural network to minimize this error. Over time, the "guesses" become incredibly accurate.

Step 6: Epsilon (epsilon) Decay

Just like in basic Q-learning, the agent starts by pressing buttons randomly (100% exploration).

- As it plays millions of frames, we decrease ϵ until it is only exploring 1% or 5% of the time.
- By the end, the agent is a "pro" that relies almost entirely on its deep-learning intuition to achieve high scores.

Output:

Untitled-2.ipynb

```python
pip install ale-py
```

[12]

```
Collecting ale-py
  Downloading ale_py-0.11.2-cp312-cp312-macosx_13_0_arm64.whl.metadata (9.0 kB)
Requirement already satisfied: numpy>1.20 in /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages (from ale-py) (2.4.1)
Downloading ale_py-0.11.2-cp312-cp312-macosx_13_0_arm64.whl (2.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.3/2.3 MB 1.7 MB/s eta 0:00:0000:0100:01
Installing collected packages: ale-py
Successfully installed ale-py-0.11.2

[notice] A new release of pip is available: 24.0 -> 26.0.1
[notice] To update, run: pip3.12 install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

[15]

```python
import random
import time
from collections import deque

import cv2
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import gymnasium as gym
from ale_py import ALEInterface          # needed for Atari ROM registration

# — Hyper-parameters ————————————————————————————————————————
ENV_NAME          = "ALE/Breakout-v5"   # swap to any Atari ROM
SEED              = 42

# Training schedule
TOTAL_STEPS       = 10_000_000          # total env steps (paper: 50M)
WARMUP_STEPS      = 50_000              # fill replay before training starts
TRAIN_FREQ        = 4                   # train every N env steps
TARGET_SYNC_FREQ  = 10_000             # hard-copy online → target every N steps
EVAL_FREQ         = 100_000             # run evaluation every N steps
EVAL_EPISODES     = 10

# Network / optimiser
BATCH_SIZE        = 32
LEARNING_RATE     = 0.0001
GAMMA             = 0.99
GRAD_CLIP         = 10.0               # clip gradient norm

# Exploration schedule
EPS_START         = 1.0
EPS_END           = 0.1
EPS_DECAY_STEPS   = 1_000_000          # linear decay over this many steps
```

Untitled-2.ipynb

```python
# — Frame preprocessing ————————————————————————————————————
def preprocess_frame(frame: np.ndarray) -> np.ndarray:
    """
    Raw Atari RGB (210×160×3)
      → grayscale
      → resize to 84×84
      → uint8  (saves ~4× memory vs float32 in the replay buffer)
    """
    gray  = cv2.cvtColor(frame, cv2.COLOR_RGB2GRAY)
    resized = cv2.resize(gray, (84, 84), interpolation=cv2.INTER_AREA)
    return resized  # shape (84, 84), dtype uint8


class FrameStack:
    """Maintains a rolling stack of the last FRAME_STACK preprocessed frames."""
    def __init__(self, k: int = FRAME_STACK):
        self.k     = k
        self._frames: deque = deque(maxlen=k)

    def reset(self, frame: np.ndarray) -> np.ndarray:
        proc = preprocess_frame(frame)
        for _ in range(self.k):
            self._frames.append(proc)
        return self._get_state()

    def step(self, frame: np.ndarray) -> np.ndarray:
        self._frames.append(preprocess_frame(frame))
        return self._get_state()

    def _get_state(self) -> np.ndarray:
        # shape: (k, 84, 84) uint8
        return np.stack(list(self._frames), axis=0)


# — Neural network ————————————————————————————————————————
class DQN(nn.Module):
    """
    3-conv + 1-FC architecture from the Nature DQN paper.
    Input:  (batch, 4, 84, 84) float32  (pixel values in [0, 1])
    Output: (batch, n_actions) Q-values
    """
    def __init__(self, n_actions: int):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(4, 32, kernel_size=8, stride=4),  # → (32, 20, 20)
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=4, stride=2), # → (64,  9,  9)
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, stride=1), # → (64,  7,  7)
            nn.ReLU(),
        )
        conv_out = 64 * 7 * 7   # = 3136
```

[15]

Untitled-2.ipynb

Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ··· Python 3.12.4

```python
        x = self.conv(x)
        x = x.view(x.size(0), -1)   # flatten
        return self.fc(x)


# — Replay buffer ————————————————————————————————————————
class ReplayBuffer:
    """
    Circular buffer storing (state, action, reward, next_state, done) tuples.
    States are stored as uint8 to save memory; converted to float on sample.
    """
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.buf: deque = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        self.buf.append((state, action, reward, next_state, done))

    def sample(self, batch_size: int):
        batch = random.sample(self.buf, batch_size)
        states, actions, rewards, next_states, dones = zip(*batch)

        # Convert to tensors
        states      = torch.from_numpy(np.stack(states)     / 255.0).float()
        next_states = torch.from_numpy(np.stack(next_states)/ 255.0).float()
        actions     = torch.tensor(actions,  dtype=torch.long)
        rewards     = torch.tensor(rewards,  dtype=torch.float32)
        dones       = torch.tensor(dones,    dtype=torch.float32)
        return states, actions, rewards, next_states, dones

    def __len__(self):
        return len(self.buf)


# — DQN Agent ————————————————————————————————————————————
class DQNAgent:
    def __init__(self, n_actions: int, device: torch.device):
        self.n_actions = n_actions
        self.device    = device

        self.online_net = DQN(n_actions).to(device)
        self.target_net = DQN(n_actions).to(device)
        self.sync_target()                      # initialise target = online

        self.optimizer  = optim.Adam(
            self.online_net.parameters(), lr=LEARNING_RATE
        )
        self.loss_fn    = nn.SmoothL1Loss()     # Huber loss
        self.replay     = ReplayBuffer(REPLAY_CAPACITY)
        self.steps_done = 0

    # — Target network sync ——————————————————————————————
    def sync_target(self):
```

[15] Python

0 0 Start JSON Server Spaces: 4 { } Cell 7 of 7

Untitled-2.ipynb

Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ··· Python 3.12.4

```python
        self.replay     = ReplayBuffer(REPLAY_CAPACITY)
        self.steps_done = 0

    # — Target network sync ——————————————————————————————
    def sync_target(self):
        """Hard copy: θ_target ← θ_online."""
        self.target_net.load_state_dict(self.online_net.state_dict())

    # — ε-greedy action selection ————————————————————————
    def select_action(self, state: np.ndarray) -> int:
        # Linear decay from EPS_START to EPS_END over EPS_DECAY_STEPS
        fraction = min(1.0, self.steps_done / EPS_DECAY_STEPS)
        eps = EPS_START + fraction * (EPS_END - EPS_START)

        if random.random() < eps:
            return random.randrange(self.n_actions)

        with torch.no_grad():
            s = torch.from_numpy(state / 255.0).float().unsqueeze(0).to(self.device)
            return int(self.online_net(s).argmax(dim=1).item())

    # — Single training step ————————————————————————————
    def train_step(self) -> float:
        if len(self.replay) < BATCH_SIZE:
            return 0.0

        states, actions, rewards, next_states, dones = [
            t.to(self.device)
            for t in self.replay.sample(BATCH_SIZE)
        ]

        # Current Q-values: Q(s, a) for the actions that were actually taken
        q_values = self.online_net(states)              # (B, n_actions)
        q_curr   = q_values.gather(1, actions.unsqueeze(1)).squeeze(1)  # (B,)

        # Target Q-values (no gradient through target network)
        with torch.no_grad():
            # Double DQN: online net selects the action, target net evaluates it
            best_actions = self.online_net(next_states).argmax(dim=1, keepdim=True)
            q_next = self.target_net(next_states).gather(1, best_actions).squeeze(1)
            q_target = rewards + GAMMA * q_next * (1.0 - dones)

        loss = self.loss_fn(q_curr, q_target)

        self.optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(self.online_net.parameters(), GRAD_CLIP)
        self.optimizer.step()

        return loss.item()


# — Evaluation ———————————————————————————————————————————
def evaluate(agent: DQNAgent, env_name: str, device: torch.device
```

[15] Python

0 0 Start JSON Server Spaces: 4 { } Cell 7 of 7

Untitled-2.ipynb

```python
# — Evaluation ————————————————————————————————————————————
def evaluate(agent: DQNAgent, env_name: str, device: torch.device,
             n_episodes: int = EVAL_EPISODES) -> float:
    """Run greedy episodes (ε=0.05 as in paper) and return mean total reward."""
    env   = gym.make(env_name)
    stack = FrameStack()
    total_rewards = []

    for _ in range(n_episodes):
        obs, _ = env.reset()
        state  = stack.reset(obs)
        episode_reward = 0.0

        while True:
            with torch.no_grad():
                s = torch.from_numpy(state / 255.0).float().unsqueeze(0).to(device)
                action = int(agent.online_net(s).argmax(dim=1).item())

            obs, reward, terminated, truncated, _ = env.step(action)
            state = stack.step(obs)
            episode_reward += reward
            if terminated or truncated:
                break

        total_rewards.append(episode_reward)

    env.close()
    return float(np.mean(total_rewards))


# — Main training loop ————————————————————————————————————
def train():
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Device: {device}")

    # Seeding
    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)

    env   = gym.make(ENV_NAME)
    stack = FrameStack()

    n_actions = env.action_space.n
    agent     = DQNAgent(n_actions, device)

    print(f"\n{'='*60}")
    print(f"  Atari DQN — {ENV_NAME}")
    print(f"  Actions : {n_actions}")
    print(f"  Network : {sum(p.numel() for p in agent.online_net.parameters()):,} params")
    print(f"  Device  : {device}")
    print(f"{'='*60}\n")
```

Untitled-2.ipynb

```python
    for step in range(1, TOTAL_STEPS + 1):
        agent.steps_done = step

        # — Select and execute action ————————————————————————
        action = agent.select_action(state)
        obs, reward, terminated, truncated, _ = env.step(action)
        done   = terminated or truncated

        next_state = stack.step(obs)

        # Reward clipping — keeps loss scale stable across different games
        clipped_reward = float(np.sign(reward))

        agent.replay.push(state, action, clipped_reward, next_state, float(done))

        state           = next_state
        episode_reward += reward

        if done:
            episode_count += 1
            ep_rewards.append(episode_reward)
            episode_reward = 0.0
            obs, _  = env.reset()
            state   = stack.reset(obs)

        # — Train ————————————————————————————————————————————
        if step >= WARMUP_STEPS and step % TRAIN_FREQ == 0:
            loss = agent.train_step()
            losses.append(loss)

        # — Sync target network ——————————————————————————————
        if step % TARGET_SYNC_FREQ == 0:
            agent.sync_target()

        # — Logging ——————————————————————————————————————————
        if step % 10_000 == 0:
            avg_reward = np.mean(ep_rewards[-50:]) if ep_rewards else 0.0
            avg_loss   = np.mean(losses[-500:])    if losses    else 0.0
            fraction   = min(1.0, step / EPS_DECAY_STEPS)
            eps        = EPS_START + fraction * (EPS_END - EPS_START)
            elapsed    = (time.time() - t_start) / 60

            print(
                f"Step {step:>9,} | "
                f"Ep {episode_count:>5,} | "
                f"Reward(50ep): {avg_reward:>7.1f} | "
                f"Loss: {avg_loss:.4f} | "
                f"ε: {eps:.3f} | "
                f"Buf: {len(agent.replay):>7,} | "
                f"{elapsed:.1f}min"
            )
```

Untitled-2.ipynb

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ... Python 3.12.4

```python
            avg_loss   = np.mean(losses[-500:])    if losses    else 0.0
            fraction   = min(1.0, step / EPS_DECAY_STEPS)
            eps        = EPS_START + fraction * (EPS_END - EPS_START)
            elapsed    = (time.time() - t_start) / 60

            print(
                f"Step {step:>9,} | "
                f"Ep {episode_count:>5,} | "
                f"Reward(50ep): {avg_reward:>7.1f} | "
                f"Loss: {avg_loss:.4f} | "
                f"ε: {eps:.3f} | "
                f"Buf: {len(agent.replay):>7,} | "
                f"{elapsed:.1f}min"
            )

        # ── Evaluation ──────────────────────────────────────────
        if step % EVAL_FREQ == 0:
            eval_reward = evaluate(agent, ENV_NAME, device)
            print(f"\n>>> Eval at step {step:,}: mean reward = {eval_reward:.1f}\n")

            # Save checkpoint
            torch.save({
                "step":   step,
                "model":  agent.online_net.state_dict(),
                "optim":  agent.optimizer.state_dict(),
            }, f"dqn_{ENV_NAME.split('/')[1]}_{step}.pt")

    env.close()
    print("\nTraining complete.")
    return agent


# ── Entry point ─────────────────────────────────────────────
if __name__ == "__main__":
    trained_agent = train()
```

[15] Python

```
Device: cpu

====================================================
  Atari DQN - ALE/Breakout-v5
  Actions : 4
  Network : 1,686,180 params
  Device  : cpu
====================================================

Step    10,000 | Ep    54 | Reward(50ep):     1.3 | Loss: 0.0000 | ε: 0.991 | Buf:  10,000 | 0.1min
Step    20,000 | Ep   108 | Reward(50ep):     1.2 | Loss: 0.0000 | ε: 0.982 | Buf:  20,000 | 0.3min
Step    30,000 | Ep   164 | Reward(50ep):     1.2 | Loss: 0.0000 | ε: 0.973 | Buf:  30,000 | 0.4min
Step    40,000 | Ep   214 | Reward(50ep):     1.6 | Loss: 0.0000 | ε: 0.964 | Buf:  40,000 | 0.6min
Step    50,000 | Ep   263 | Reward(50ep):     1.6 | Loss: 0.0012 | ε: 0.955 | Buf:  50,000 | 0.7min
Step    60,000 | Ep   313 | Reward(50ep):     1.8 | Loss: 0.0041 | ε: 0.946 | Buf:  60,000 | 4.7min
Step    70,000 | Ep   365 | Reward(50ep):     1.3 | Loss: 0.0036 | ε: 0.937 | Buf:  70,000 | 21.0min
```

0 ⚠ 0 Start JSON Server Spaces: 4 { } Cell 7 of 7